nvim-clipboard

Uma área de transferência dedicada a cada projeto específico.

Funcionalidades

- ☒ Respeita repositórios Git
- ☒ Integração com Telescope
- ☒ Fácil lista de visualização e edição

Como instalar:

Lazy

```
return {
    "EnderMega/meuclipboard",
    event = { "BufNewFile", "BufReadPre" }, -- <- Opicional

    config = function()
        local meuclip = require("meuclipboard")
        meuclip.setup()
    end
}
```

Atalhos padrão

Atalho	Descrição	Comando
<leader>,s	Escreve o que deseja salvar no prompt do neovim	`meuclip.salvar`
<leader>,S	Escreve o que deseja salvar em uma caixa flutuante (Requer *dressing.nvim* ou *snacks.nvim*)	`meuclip.salvarP`
<leader>„	Pede o index do texto para ser colado	`meuclip.colar`
<leader>,.	Repete a última colagem (Não repete o último count)	`meuclip.colar(true, false)`
<leader>,E	Abre o arquivo da área de transferência para editar	`meuclip.editar`
<leader>,r	Remover o item no index passado no prompt do neovim	`meuclip.remover`

Atalho	Descrição	Comando
<leader>,l	Abre o arquivo da área de transferência na linha indicada no prompt do neovim	`meuclip.editar(tonumber(vim.fn.input({ prompt = "Index: "}))`
<leader>,e	Mostra todos os itens em uma lista flutuante	`meuclip.mostrarLista`
<leader>,t	Mostra todos os itens no Telescope, *com o tema que eu escolhi* (Requer *telescope*)	`meuclip.procurarTelescope(require("telescope.t`

Customização

`setup()` recebe uma mesa que pode receber qualquer um dos seguintes argumentos, os especificados na frente são o que vem por padrão:

```
meuclip.setup({
    editar_padrao = false,
    colar_depois = false,
    colar_mover_cursor = false,
    nova_linha = false,
    altura_lista = nil,
    novo_count = 1,
    verificar_pasta = false,

    maps = nil      -- { ... }
    merge_maps = false,
    map_opts = nil  -- { ... }
})
```

- editar_padrao [boolean]: Se ao passar um index invalido (index < 0) para `meuclip.editar` ele vai para a linha 1 `(true)` ou não `(false)`.
- colar_depois [boolean]: `vim.api.nvim_put(~, ~, after, ~)`: `(true)` age como 'p', `(false)` age como 'P'.
- colar_mover_cursor [boolean]: `vim.api.nvim_put(~, ~, ~, follow)`: Se verdadeiro coloca o cursor no fim do texto inserido.
- nova_linha [boolean]: Se ao colar múltiplas vezes com `count` deve colar cada um e um nova linha.
- altura_lista [nil|integer]: Em vez de a altura da lista ser definida pela quantidade de elementos ela pode ser definida para ter uma altura fixa.
- novo_count [integer]: > 0 => Faz o `pre_count` ser sempre o último count passado: `pre_count` = y, count = x, colar(~, true|false); `pre_count` = x 1 => Se repetir_ultimo_count == true e *passar* count, usa o valor do count como o novo `pre_count`: `pre_count` = y, count = 0 (sem count), colar(~, true); pre_count = y `pre_count` = y, count = x, colar(~, true);

pre_count = x (count = 1 incluido) `pre_count` = y, count = x, colar(~, false); pre_count = x (count = 1 incluido) 2 => Se repetir_ultimo_count == true, sempre mantém mantém o valor do `pre_count` `pre_count` = y, count = x (sem count, 0, e 1 incluido), colar(~, true); pre_count = y

- verificar_pasta [boolean]: Se o plugin deve criar um autocmd que verifica se a pasta de trabalho do neovim mudou e se referir a área de transferencia da nova pasta.
- maps [dictionary]: Um dicionário com uma chave, que é o nome do atalho, seguido do seu valor que é uma string com o atalho. `{ aaa = "...", bbb = "...", ... }`.
- merge_maps [boolean]: Se deve mesclar `(true)` ou substituir os atalhos padrãos com os do usuário `(false)`.
- maps_opts [dictionary]: As opções para vim.keymap.set(~, ~, ~, map_opts), como { desc = "~", remap = true, silent = true }.

If you are able to, consider donating:

